Can Agent Benchmarks Support Their Scores?
Evidence-Supported Bounds for Interactive-Agent Evaluation

Anonymous Author(s)
Affiliation
Address
email

Abstract

Interactive-agent benchmarks map an agent run to a binary outcome via outcome checks. When checks rely on surface-level signals or miss the agent's action path, they fail to determine whether the run succeeded. For example, a benchmark case asks whether Alice's shipping address changed, but its outcome check only verifies that the agent clicked `Save`. This does not guarantee that the intended state change was applied; the agent may have updated the wrong record. Counting such a run as a success makes the score misleading. Benchmark quality therefore depends on outcome detection as well as task design.

We address this by adding an outcome-evidence reporting layer to existing benchmarks, without changing their tasks, agents, or evaluators. The layer does three things: *(i)* specifies, before scoring, which stored artifacts would verify the claimed outcome for each case; *(ii)* applies the locked case checklist to each completed record and assigns one of three evidence states: *supported success*, *supported failure*, or *unresolved*; and *(iii)* reports evidence-supported bounds that quantify uncertainty from unresolved records. Records that remain *unresolved* are kept visible rather than silently counted, dropped, or hidden inside a single success rate.

We evaluate this outcome-evidence layer on four public benchmarks: AGENTDOJO, τ^3-bench retail, APPWORLD, and WEBARENA-VERIFIED. The resulting reports make outcome-evidence quality visible alongside task quality, supporting more evidence-grounded agent evaluation and uncertainty reporting. We release the case checklists, reason labels, audit checks, robust-ranking summaries, and scoring script needed to reproduce the reports.

1 Introduction

Interactive-agent benchmarks increasingly ask agents to navigate interfaces, call tools, and change persistent environment state. Their headline result is usually a single success rate. But a reported success is only as reliable as the evidence used to verify it. If a task asks an agent to update Alice's shipping address, a trace showing that the agent clicked `Save` is not the same as evidence that Alice's backend record changed. The same action trace is compatible with the correct record being updated, the wrong record being modified, or no durable state change occurring at all.

This is an outcome-evidence gap. The benchmark claim is a binary statement about what happened in the environment, while the stored artifacts may include only screenshots, action logs, final messages, partial verifier inputs, or other indirect traces. The gap is especially important for interactive agents because correctness often depends on identity resolution, side effects, post-state queries, policy constraints, and multi-step tool interactions. When these artifacts are insufficient, a point score can overstate what the benchmark has actually verified.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

We address this gap with an outcome-evidence reporting layer. The layer is additive: it leaves benchmark tasks, agents, environments, and native evaluators unchanged. For each case, it writes a short list of evidence needed before scoring, specifying which stored artifacts would decide the benchmark's own success claim. Each completed record is then reported as *supported success*, *supported failure*, or *unresolved*. An *unresolved* record is not treated as a success, not converted into a failure, and not dropped from the evaluation set.

When a record is unresolved, none of the usual shortcuts is neutral. Counting it as a success overstates what the evidence supports. Counting it as a failure can punish missing instrumentation rather than agent behavior. Dropping it changes which records are evaluated and can create agent-dependent post-selection, because some agents leave more decisive traces than others. We therefore keep unresolved records in the same set of completed records and report the range of all-record performance still compatible with the stored evidence. The counted-only score is still useful, but only as a diagnostic conditional on decidability, not as a headline all-record estimate.

We evaluate this reporting layer on four public interactive-agent benchmarks. AGENTDOJO and τ^3-bench retail provide evidence-sensitive settings where plausible traces can outrun stored evidence. APPWORLD and WEBARENA-VERIFIED use strong state-based or verified evaluators and serve as calibration controls. The study is designed to be falsifiable: large unresolved shares on the controls would indicate over-conservative evidence requirements, while narrow control bounds and wider evidence-sensitive bounds would show that the layer distinguishes weak stored evidence from strong native verification.

The paper makes four contributions: *(i)* we formalize the outcome-evidence gap in interactive-agent benchmarks, where a binary success claim may be reported even when stored artifacts are insufficient to decide that claim; *(ii)* we introduce case checklists tied to each benchmark's own success claim, plus a three-state counting rule that separates *supported success*, *supported failure*, and *unresolved* records without changing benchmark tasks, agents, environments, or native evaluators; *(iii)* we derive evidence-supported performance bounds over the fixed set of completed records, making the counted-only score a diagnostic over decidable records rather than a headline all-record estimate; and *(iv)* we apply the method to four public benchmarks, showing that evidence uncertainty is concentrated in evidence-sensitive domains while remaining narrow on strong state-based evaluator controls, and release the case checklists, reason labels, audit checks, robust-ranking summaries, and scoring script needed to reproduce the reports.

2 Related Work and Preliminaries

Interactive-agent evaluation and outcome checks. Interactive-agent benchmarks evaluate agents that navigate interfaces, call tools, and alter environment state. Relevant environments include WebArena [17], BrowserGym [1], ToolBench-style tool-use benchmarks [9], AGENTDOJO [2], τ^3-bench retail [12], ANDROIDWORLD [10], WORKARENA [3], and OSWORLD-VERIFIED [14]. These benchmarks make interactive evaluation realistic, but realism alone does not guarantee that a stored trace decides the outcome claim being reported.

Verified evaluators and benchmark validity. A recent line of work strengthens outcome checks through state-based or verified evaluation, including the verified WEBARENA-VERIFIED release [5], APPWORLD database-state unit tests [13], and SWE-bench Verified [8]. Benchmark-validity work, including the Agentic Benchmark Checklist, argues that construct validity, outcome validity, and reporting assumptions should be made explicit [18]. Our contribution is complementary: rather than proposing a new environment, a stronger domain evaluator, or an audit checklist, we add a cross-benchmark reporting layer that asks what the stored artifacts of completed runs can support under each benchmark's own claim. LLM-as-judge methods [16, 11] and benchmark refresh or contamination studies [6, 4, 15] are useful diagnostics, but they do not replace evidence that a native outcome claim is decidable from stored artifacts.

Claims, records, traces, and evidence. A *case* specifies a task, the initial environment, the benchmark claim being tested, and the evidence required to decide that claim. A benchmark claim is the binary outcome statement the benchmark reports for a completed run, such as whether a target record was updated, a message was sent, or a constraint was violated. The *native evaluator* is the benchmark's released mechanism for turning its available artifacts into a success or failure label.

We use *case unit* only for a small bookkeeping issue: in most domains, one task instance is one case unit, while in paired-arm settings such as AGENTDOJO, two runnable arms form one case unit even though they are executed separately. Decisions live at the level of *records*. A record is one agent's result on one case unit. The distinction matters because the same case unit can be countable for one agent, whose trace exposes decisive evidence, and `UNRESOLVE` for another, whose trace does not.

An *episode* is one concrete execution of one agent on one runnable arm of a case. Episodes produce *traces*: ordered records of actions, observations, tool calls, browser events, messages, files, and post-run artifacts. We use *stored artifacts* for the full set of traces, logs, state snapshots, receipts, verifier inputs, and post-run measurements retained after execution. *Evidence* is the subset of those artifacts that can support the active benchmark claim. A trace can be plausible without being decisive.

Staying tied to the benchmark claim. Because the method sits on top of existing benchmarks, the requirements must not silently make the task harder. For each case unit, we write down: (i) the benchmark's own binary claim; (ii) the official source of that claim, such as the task text, evaluator code, policy, or schema; (iii) the artifacts needed to decide success or fail; (iv) the rule for assigning `UNRESOLVE`; and (v) whether any requirement goes beyond the benchmark's own claim.

The requirements must be minimal with respect to the benchmark's own semantics. For example, if a task says only "update the address" and the native evaluator only checks the target record's address, then the requirements ask for target-record identity and post-state evidence. A no-collateral-change requirement is included only if the benchmark's own task, policy, evaluator, or reported claim requires it. Otherwise it is labeled as a stronger measurement claim and reported separately.

When sources differ, the requirements follow a fixed source hierarchy: official evaluator semantics first; official task text and policy second; and schema constraints needed to interpret evaluator-visible state third. We do not add requirements from annotator intuition alone. Any requirement not grounded in one of these sources is labeled as a stronger measurement claim and excluded from the main bounds tied to the benchmark's own claim.

3 Method

The method turns completed benchmark runs into an evidence-supported report. It keeps the benchmark's native view, namely the released evaluator's pass/fail label, and adds an evidence view that asks whether the stored artifacts are sufficient to decide the same underlying outcome claim. The layer is post-run with respect to agent execution, but fixed before evidence scoring: it does not change the task, prompt, agent, environment, native evaluator, or native score.

Case checklist. For each case unit, we first make the outcome claim explicit. An LLM-assisted drafter reads the user goal, task text, official policy when present, released evaluator or oracle code, and relevant state schemas. It proposes a compact case checklist: what the user is trying to achieve, what condition the benchmark reports as success, which official code or oracle checks that condition, which artifacts would decide it after execution, and when the record should remain undecided. Human review source-checks each checklist against the hierarchy in Section 2. The checklist is not a new task or answer key; it states what stored evidence would be sufficient to verify the benchmark's own claim. Conditions that go beyond that claim are labeled as stronger measurement claims and reported separately rather than used in the native-aligned headline bounds.

Evidence scoring. We then run the benchmark normally and retain the artifacts available for review: traces, tool calls, evaluator inputs and outputs, final messages, and saved database, browser, file, or API state. For each completed record, the native view records the released evaluator's label. The evidence view applies the locked case checklist to the stored artifacts and assigns one of three states: S, if the artifacts support success under the benchmark claim; F, if they support failure; and U, if they do not decide the claim. Agent-caused invalid actions, timeouts, tool misuse, or aborts after task start are failures when the released benchmark would fail the run; they are not moved into U. Thus the evidence view is an additional report over the same runs, not a replacement for the native benchmark score.

Counting rule. The reporting rule uses only the three evidence-state counts. For agent a in domain d, let $N_{a,d}$ be the number of completed records in the evaluation set, and let $S_{a,d}$, $F_{a,d}$, and $U_{a,d}$

be the counts of the three states, so $N_{a,d} = S_{a,d} + F_{a,d} + U_{a,d}$. The counted-only score answers a conditional question: among records the stored artifacts decide, what fraction are successes? The bound answers the all-record question: over every completed record, what performance values are still compatible with the stored evidence? Writing S, F, U, N for the counts in a given agent-domain cell, we report

$$\mathrm{CountedScore}_r(a,d) = \frac{S}{S+F}, \quad \mathrm{Perf}_r(a,d) \in \left[\frac{S}{N}, \frac{S+U}{N}\right], \quad \mathrm{Width}_r(a,d) = \frac{U}{N}.$$

The lower endpoint counts only supported successes; the upper endpoint treats every unresolved record as a possible success. The width is exactly the unresolved share. When $U = 0$, the bound collapses to the all-record score. When $U > 0$, a single point estimate must make an unobserved decision about records the artifacts did not decide. These intervals are not confidence intervals; they are partial-identification bounds describing what the stored evidence can and cannot identify [7]. The counted-only score remains useful as a diagnostic conditional on decidability, but the headline object is the evidence-supported bound.

Comparisons and diagnostics. We keep three diagnostics separate from the headline bounds. First, case-cluster bootstrap intervals describe sensitivity to the finite set of sampled cases. Each replicate resamples case units within a domain and keeps all agent records attached to the resampled case unit. These intervals condition on the stored traces and fixed agent versions; they do not measure uncertainty from repeated executions, API drift, tool nondeterminism, or environment volatility.

Second, pairwise rankings are supported only when the bounds separate. For agents i and j in domain d, the bound-implied difference interval is $\Delta(i,j;d) = [\mathrm{LB}_{i,d} - \mathrm{UB}_{j,d}, \mathrm{UB}_{i,d} - \mathrm{LB}_{j,d}]$. We report $i \succ j$ only if $\mathrm{LB}_{i,d} > \mathrm{UB}_{j,d}$, and $j \succ i$ only if $\mathrm{LB}_{j,d} > \mathrm{UB}_{i,d}$. Otherwise the pair is not identified and is marked "?" rather than a tie. For a separated ordering $i \succ j$, the dominance margin is $M_{i>j} = \mathrm{LB}_{i,d} - \mathrm{UB}_{j,d}$. We add an asterisk only when the 2.5th percentile of the case-cluster bootstrap distribution of $M_{i>j}$ remains positive; the asterisk is a descriptive stability tag, not a formal significance test.

Third, every U record receives one reason, so the bound width can be audited. The reasons are: missing state query (needed backend or environment state was not saved), unobservable side effect (the consequence of an action is not exposed), ambiguous identity mapping (the acted-on record or recipient is not unique), missing state-preservation evidence (the native claim requires unrelated state to remain unchanged but no diff or snapshot is available), paired-arm asymmetry (one arm of a paired case is decidable and the other is not), evaluator-output ambiguity (the released evaluator emits a label but the stored inputs do not determine it), and claim-scope mismatch (the trace supports a related task but not the reported claim). Appendix C gives representative examples and the priority rule used when more than one reason could apply.

4 Experiments

We evaluate whether the reporting layer changes what benchmark scores can claim, not which model wins a leaderboard. The experiment fixes four public interactive-agent benchmarks, three current frontier agents, and a 100-case sample per benchmark. Each run is evaluated twice: once by the benchmark's native evaluator, and once by the evidence-supported reporting layer defined in Section 3.

4.1 Benchmarks

The benchmark suite contains two strong-evaluator controls and two domains where the native claim can depend on evidence that is not always stored. This mix is important: a useful reporting layer should leave strong native verification mostly narrow while exposing uncertainty when traces are insufficient.

4.2 Agents and workload

We run three agents: OpenAI GPT-5.4, Claude 4.6, and DeepSeek V4 Pro. The paper does not treat these models as an exhaustive comparison set. They are fixed measurement probes used to test

Table 1: Benchmark suite. We sample 100 case units from each benchmark and run three agents on each case unit, yielding 300 records per benchmark. AGENTDOJO uses paired benign/injected arms, so its 300 records correspond to 600 execution episodes.

Benchmark	Role	Native outcome signal	Stored evidence used
τ^3-bench retail	Retail tool-use benchmark with identity and policy-sensitive backend state.	native success signal	database state; tool records; policy-relevant backend evidence
APPWORLD	Strong-evaluator control with database-state unit tests.	database-state unit tests	database state; API logs; unit-test artifacts; native field checks
WEBARENA-VERIFIED	Strong-evaluator control with verified web-task checks.	verified deterministic evaluator	browser artifacts; network trace; structured final output; verifier inputs
AGENTDOJO	Evidence-sensitive domain with paired utility and security claims.	utility/security label	workspace state across both arms; tool calls; files; messages

whether outcome-evidence uncertainty is a property of benchmark evidence rather than an artifact of one agent family. The release records concrete model identifiers, API settings, prompts, tool wrappers, and run dates.

For each benchmark, the manifest selects 100 eligible case units using a deterministic hash order after excluding smoke-test and malformed cases. Each case unit is run once per agent, giving $4 \times 100 \times 3 = 1200$ completed record slots before infrastructure exclusions. AGENTDOJO executes two arms per record, so the main experiment stores 1500 execution episodes in total: 600 for AGENTDOJO and 300 for each of the other three benchmarks.

4.3 Execution and evidence scoring

Every agent run uses the benchmark's original task interface, tools, environment, and released evaluator. We store the native evaluator output alongside traces, tool calls, final messages, evaluator inputs, and available post-run state. The native score is therefore recoverable without the evidence-supported layer.

Before evidence scoring, an LLM-assisted drafter produces one case checklist per case unit. The drafter receives the user goal, task text, official policy when present, evaluator code or description, relevant database/API/browser/file/tool schemas, and the schema of artifacts saved after the run. Two researchers cross-validate the checklists in two review rounds. They check whether each checklist follows the source hierarchy in Section 2; checklist items that fail are manually edited or regenerated and reviewed again. Once locked, the checklists cannot be changed based on agent outcomes.

Denominator rule. Only infrastructure or pre-run failures are excluded from the evaluation set. Examples include a sandbox that fails to boot before the task is presented, a missing benchmark asset, or an external service outage that prevents normal benchmark execution. Agent-caused invalid actions, tool misuse, timeouts after task start, malformed final answers, or benchmark-facing aborts are not excluded. They are counted as `FAIL` when the native benchmark would fail the run, and they are also reported in the all-attempt audit in Table 7. This separation prevents evidence filtering from raising scores by removing difficult or agent-induced failures.

Sample size and precision. The primary unit for uncertainty is the case unit, not the record. Each benchmark has 100 case units crossed with three agents, and the three records attached to a case unit can be correlated because the agents face the same task. We therefore avoid formal precision claims based on 300 independent records per benchmark. A record-level binomial calculation would be, at best, an independence lower bound and is not used for inference. All resampling intervals are case-clustered and condition on the stored traces.

Table 2: Main measurement report. Counts are over completed records after excluding only infrastructure/pre-run failures. Native score is the released benchmark score on the same completed records. Coverage is the share of records that stored artifacts decide. Counted-only score is conditional on those decidable records. Lower and Upper are the all-record evidence-supported endpoints; Width is the unresolved share.

Benchmark	Total	Native score	SUCCESS	FAIL	UNRESOLVE	Coverage	Counted-only score	Lower	Upper	Width
τ^3-bench retail	300	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
AppWorld	300	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
WebArena-Verified	300	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
AgentDojo	300	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
All benchmarks	1200	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL

Placeholder: native score and evidence-supported endpoints.

Final figure: one row per benchmark and model. Plot the native score as a marker, the counted-only score as a smaller diagnostic marker, and the evidence-supported interval $[\mathrm{Lower}, \mathrm{Upper}]$ as a horizontal line. Sort benchmarks by Width so that evidence-sensitive domains appear first.

Figure 1: Planned visual summary of Table 2 at the model-by-benchmark level. The figure should make the paper's main empirical pattern visually falsifiable: strong state-based benchmarks should have narrow intervals, while evidence-sensitive benchmarks may have wide intervals.

4.4 Audit checks

We run two checks for inspectability. First, two researchers audit 100 records, 25 per benchmark. The sample is stratified within each benchmark to include decidable records, `UNRESOLVE` records, and native/evidence disagreement records when all three buckets are present. Auditors see the task, trace, available artifacts, and locked checklist, but not the agent identity, native label, evidence label, or reason tag.

Second, we rerun a fixed reference agent, OpenAI GPT-5.4, on ten case units per benchmark. This rerun subset checks whether evidence labels are stable under a small repeated-execution probe. It is not used as an uncertainty interval for agent performance.

Adoption note. Adopting the method on top of an existing benchmark requires writing one fixed claim and one case checklist per case unit, then running the scoring script over stored traces. The script returns S, F, U, coverage, counted-only score, lower and upper bounds, bound width, and `UNRESOLVE`-reason records. Per-benchmark human-time breakdowns appear in Appendix E.

5 Evaluation

The evaluation is organized around the questions a reader of an agent benchmark actually needs answered. What did the original benchmark report? How much of that report is directly supported by stored artifacts? If support is missing, how large is the resulting uncertainty, where does it come from, and does it change model comparisons? The tables below are intentionally fillable from the scored manifest: they specify exactly what evidence a final benchmark report should expose.

5.1 Evaluation 1: Evidence support for reported scores

The first result is the score report itself. Table 2 keeps the native benchmark score visible, then adds the evidence view: supported successes, supported failures, unresolved records, coverage, the counted-only score, and the lower and upper endpoints. This table should let a reader tell the difference between two very different situations: a native score that is backed by nearly complete stored evidence, and a native score whose apparent precision comes from treating unresolved records as if they had been decided.

Table 3: Main unresolved-evidence diagnostic. Width is the unresolved share from Table 2. The top reason identifies the largest source of missing evidence; the final column should be filled with the concrete benchmark repair suggested by that reason.

Benchmark	Width	Top unresolved reason	Benchmark repair suggested by the evidence gap
τ^3-bench retail	FILL	FILL	FILL
AppWorld	FILL	FILL	FILL
WebArena-Verified	FILL	FILL	FILL
AgentDojo	FILL	FILL	FILL

Table 4: Directional pairwise robust-ranking matrix. Three agents induce three pairwise comparisons per benchmark. A cell $A > B$ is reported only when $\mathrm{LB}_{A,d} > \mathrm{UB}_{B,d}$; ? denotes bound overlap. An asterisk marks unadjusted case-resampling stability, not statistical significance. It is added only when the observed bounds are separated and the lower 2.5th percentile of the case-cluster bootstrap dominance margin remains positive.

Benchmark	GPT-5.4–Claude 4.6	GPT-5.4–DeepSeek V4 Pro	Claude 4.6–DeepSeek V4 Pro	Non-identified pairs
τ^3-bench retail	?	?	?	FILL
AppWorld	?	?	?	FILL
WebArena-Verified	?	?	?	FILL
AgentDojo	?	?	?	FILL
All benchmarks	Unsupported native strict orderings: FILL; native conflicts: FILL			FILL

5.2 Evaluation 2: Counted-only scores are conditional on decidability

The counted-only score answers a narrow but useful question: among records where stored artifacts decide the benchmark claim, how often is success supported? Its denominator is therefore not the full evaluation set when unresolved records exist. In the final analysis, a high counted-only score with low coverage should be described as "high success on the decidable subset," not as strong all-record performance. Case-resampling intervals for the same quantities appear in Appendix D.

The next diagnostic asks why records are unresolved. Table 3 should not be read as a new score; it is a repair map for benchmark authors. If most unresolved records come from missing state queries, the fix is better post-run state capture. If they come from identity ambiguity, the fix is unique identifiers or stricter task schemas. If they come from claim-scope mismatch, the benchmark needs a clearer statement of what its score claims.

5.3 Evaluation 3: Model rankings require separated bounds

The third result asks whether leaderboard comparisons survive unresolved records. Table 4 reports a directional ordering only when one model's lower endpoint exceeds another model's upper endpoint. A question mark means the ordering is not identified from stored evidence; it is not a tie. This is deliberately stricter than comparing native point scores, because the point scores may differ only inside the unresolved part of the evaluation set.

5.4 Evaluation 4: Counting decisions are inspectable

The fourth result checks that the labels are not just another opaque judge. The blinded audit samples counted records, unresolved records, and native/evidence disagreements. The rerun subset checks whether labels remain stable on a small repeated-execution probe. Disagreements are useful: they identify cases where the requirement text, artifact capture, or reason taxonomy needs clarification.

6 Discussion

The central empirical interpretation should be filled from Table 2 and Figure 1. In the final version, this paragraph should state which benchmark has the largest evidence gap, whether the strong-evaluator

Table 5: Stratified audit and rerun checks. Audit strata report how many items come from counted records, records labeled UNRESOLVE, and native/evidence disagreement records. Agreement and κ values are descriptive for this stratified stress sample, not unweighted estimates for the full record population. Taxonomy κ uses {R1–R7} and is computed only on items both auditors marked UNRESOLVE. Rerun agreement is a small stability check for evidence labels, not an agent-performance uncertainty estimate.

Benchmark	Audit items	Audit strata	Audit agreement	Countability κ	Taxonomy κ	Rerun agreement / dominant disagreement
τ^3-bench retail	25	FILL	FILL	FILL	FILL	FILL; FILL
APPWORLD	25	FILL	FILL	FILL	FILL	FILL; FILL
WEBARENA-VERIFIED	25	FILL	FILL	FILL	FILL	FILL; FILL
AGENTDOJO	25	FILL	FILL	FILL	FILL	FILL; FILL
All benchmarks	100	FILL	FILL	FILL	FILL	FILL; FILL

controls have narrow bounds, and whether native scores fall inside the evidence-supported endpoints: FILL. This is the main falsification check for the paper. If strong state-based controls have wide bounds, the case checklists are too conservative or the stored artifacts are weaker than expected; if evidence-sensitive domains have wide bounds while controls remain narrow, the method is detecting a real outcome-evidence difference rather than mechanically lowering scores.

The method is deliberately narrower than a new evaluator. It does not change tasks, prompts, environments, agents, or the native score. Its claim is that a benchmark report should distinguish what the released evaluator counted from what stored artifacts can support. The discussion of rankings should therefore be filled from Table 4: FILL. A non-identified ordering is not a tie; it means the stored evidence supports more than one completion of the unresolved records.

The bounds are also not confidence intervals. They are induced by unresolved evidence. Case-cluster bootstrap intervals answer a separate question about finite-case resampling conditional on stored traces and fixed agent versions. The audit and rerun checks should be summarized here once filled: FILL. Their role is inspectability, not a repeated-execution performance estimate.

The main limitations follow from the design. A vague benchmark claim cannot be rescued by a cleaner report; the seven-category UNRESOLVE taxonomy is a study vocabulary rather than a universal ontology; three agents are measurement probes rather than a definitive leaderboard; and stronger checks must remain separate from the native claim. The practical implication is direct: benchmark reports should include supported successes, supported failures, unresolved records, coverage, counted-only score, lower and upper endpoints, width, and reason labels.

7 Conclusion

Interactive-agent benchmarks often report scores over executed tasks, but execution is not the same as evidence-supported measurement. We introduced a reporting layer tied to each benchmark's own claim: it classifies completed records as supported success, supported failure, or UNRESOLVE, then reports the evidence-supported bounds implied by unresolved records. Across AGENTDOJO, APPWORLD, WEBARENA-VERIFIED, and τ^3-bench retail, the evaluation asks whether uncertainty is visible where stored artifacts do not decide the benchmark claim and remains narrow where native evaluators already provide strong state-based evidence. Future benchmark reports should include supported successes, supported failures, unresolved records, and bounds, so that a score is read together with the evidence that can actually support it.

References

[1] Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. The browsergym ecosystem for web agent research, 2024. URL https://arxiv.org/abs/2412.05467.

[2] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024. URL https://arxiv.org/abs/2406.13352.

[3] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks?, 2024. URL https://arxiv.org/abs/2403.07718.

[4] Batu Guan, Xiao Wu, Yuanyuan Yuan, and Shaohua Li. Is your benchmark (still) useful? dynamic benchmarking for code language models, 2025. URL https://arxiv.org/abs/2503.06643.

[5] Amine El Hattami, Megh Thakkar, Nicolas Chapados, and Christopher Pal. Webarena verified: Reliable evaluation for web agents. SEA @ NeurIPS 2025 poster, 2025. URL https://openreview.net/forum?id=94tlGxmqkN. OpenReview non-archival submission.

[6] Douwe Kiela, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, Grusha Prasad, Amanpreet Singh, Pratik Ringshia, Zhiyi Ma, Tristan Thrush, Sebastian Riedel, Zeerak Waseem, Pontus Stenetorp, Robin Jia, Mohit Bansal, Christopher Potts, and Adina Williams. Dynabench: Rethinking benchmarking in NLP. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4110–4124. Association for Computational Linguistics, 2021. URL https://aclanthology.org/2021.naacl-main.324/.

[7] Charles F. Manski. *Partial Identification of Probability Distributions*. Springer, 2003.

[8] OpenAI. Introducing SWE-bench Verified. Blog post, 2024. URL https://openai.com/index/introducing-swe-bench-verified/. Accessed: 2026-05-01.

[9] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. URL https://arxiv.org/abs/2307.16789.

[10] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents, 2025. URL https://arxiv.org/abs/2405.14573.

[11] Lin Shi, Chiyu Ma, Wenhua Liang, Xingjian Diao, Weicheng Ma, and Soroush Vosoughi. Judging the judges: A systematic study of position bias in llm-as-a-judge, 2024. URL https://arxiv.org/abs/2406.07791.

[12] Sierra Research. τ^3-Bench: Advancing Agent Benchmarking to Knowledge and Voice. Research release, 2026. URL https://sierra.ai/resources/research/tau-3-bench. Accessed: 2026-04-30.

[13] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents, 2024. URL https://arxiv.org/abs/2407.18901.

[14] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL https://arxiv.org/abs/2404.07972.

[15] Shuo Yang, Wei-Lin Chiang, Lianmin Zheng, Joseph E. Gonzalez, and Ion Stoica. Rethinking benchmark and contamination for language models with rephrased samples, 2023. URL https://arxiv.org/abs/2311.04850.

[16] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. URL https://arxiv.org/abs/2306.05685.

[17] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2023. URL https://arxiv.org/abs/2307.13854.

[18] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks, 2025. URL https://arxiv.org/abs/2507.02825.

A Experiment Manifest

The appendix now contains only material needed to audit the claims in the main paper: the workload manifest, the case-level checklist used for scoring, additional result tables, the human audit protocol, and release requirements. Exploratory domains, judge-only diagnostics, benchmark-maintenance experiments, and duplicate formal notation are intentionally omitted from the main appendix until the corresponding experiments are complete.

Table 6: Main experiment manifest. Each benchmark contributes 100 sampled case units evaluated by three agents. For AGENTDOJO, a record contains the paired benign/injected arms needed for the benchmark's native claim, so the episode count is twice the record count.

Benchmark	Case units	Agents	Records	Episodes	Stored artifacts expected for scoring
τ^3-bench retail	100	3	300	300	Tool traces, policy-relevant backend state, customer/order identifiers, native evaluator output
APPWORLD	100	3	300	300	Database snapshots, unit-test inputs and outputs, tool traces, native evaluator output
WEBARENA-VERIFIED	100	3	300	300	Browser trace, verifier inputs and outputs, relevant page or backend state, native evaluator output
AGENTDOJO	100	3	300	600	Paired benign/injected traces, tool-call logs, security-policy state, native evaluator output
All benchmarks	400	3	1200	1500	Case checklist, stored artifacts, native label, evidence label, reason label when unresolved

Table 7: Denominator audit. Only infrastructure or pre-run failures are excluded before forming the completed evaluation set. Agent-caused aborts, invalid actions, and timeouts after normal benchmark execution starts remain in the denominator and are counted as failures when the native benchmark would fail them.

Domain	Attempted	Infra/pre-run excluded	Completed	Agent-caused FAIL	Notes
AGENTDOJO	FILL	FILL	300	FILL	Paired benign/injected arms remain linked at the case-unit level.
APPWORLD	FILL	FILL	300	FILL	Native database-state failures remain in the evaluation set.
WEBARENA-VERIFIED	FILL	FILL	300	FILL	Browser-task timeouts after start remain in the evaluation set.
τ^3-bench retail	FILL	FILL	300	FILL	Tool misuse and policy-path failures remain in the evaluation set.
All domains	FILL	FILL	1200	FILL	Exclusions are not allowed to depend on whether the agent likely succeeded.

B Case Checklists and Scoring Inputs

Before scoring, we create one fixed case checklist per sampled case unit. The checklist is drafted with an LLM and then reviewed by two researchers in two cross-validation rounds. The draft records the user's goal, what the official benchmark oracle or evaluator checks, what actions or state changes are needed for completion, and which stored artifacts would let a scorer decide the benchmark's own claim. If a draft item is unsupported, ambiguous, or stronger than the native claim, reviewers edit it or regenerate that item and review it again. The checklist is locked before agent identities, agent traces, native scores, or evidence labels are shown to the scorer.

After each run, we collect the stored artifacts and score the record twice: the native benchmark evaluator gives the official/native label, and our scorer uses the locked checklist to assign supported success, supported failure, or `UNRESOLVE`. This separation is important: the paper reports both what the original benchmark would have counted and what the stored artifacts can support under the additional case checklist.

Table 8: Fields in the locked case checklist. The checklist is a plain-language scoring aid, not a new benchmark task.

Field	Purpose
User goal	Restates the task in benchmark language, including the intended object or recipient when available.
Native success claim	States what the released benchmark score claims to measure for this case.
Official oracle or evaluator	Points to the official unit test, verifier, reward function, policy rule, or evaluator output used by the benchmark.
Completion requirements	Lists the minimal state changes or side effects needed for the native claim to be true.
Evidence needed	Lists stored artifacts that can verify the claim, such as state diffs, database rows, receipts, verifier inputs, or unique identifiers.
Success/failure/unresolved rule	Gives the scorer a deterministic rule for assigning the three labels from stored artifacts.
Stronger-check flag	Separates useful but non-native checks from the benchmark-claim result.
Review notes	Records reviewer edits, regeneration reason, and final approval in two cross-validation rounds.

Table 9: Case-checklist drafting metadata. These values are filled from the scored manifest and released with the review package.

Field	Value
Drafter model	FILL
Temperature	FILL
Prompt version	FILL
Visible inputs	Task text; official policy, if any; evaluator code or evaluator description; database/API/browser/file/tool schema; trace schema; available post-run artifact types
Hidden inputs	Agent identity; agent trace; native score; outcome label; alternate-view verdicts
Human review rule	Two researchers cross-validate every checklist in two rounds before scoring. Unsupported requirements are edited or regenerated, then rechecked. Requirements that go beyond the benchmark claim are kept separate.
Source hierarchy	Evaluator semantics first; task text and policy second; schema constraints needed to interpret evaluator-visible state third.

Listing 1: Compact checklist-generation prompt skeleton. The released prompt version contains benchmark-specific input fields and formatting constraints.

```
Draft a case checklist for evidence-supported scoring.

Inputs:
- task text
- official policy, if any
- official oracle, evaluator code, or evaluator description
- relevant database/API/browser/file/tool schema
- trace schema and available post-run artifacts

Rules:
1. State the user's goal and the benchmark's native success claim.
2. List the official oracle or evaluator signal.
3. List the minimal completion requirements implied by the native claim.
4. List the stored artifacts needed to verify those requirements.
5. Define SUCCESS, FAIL, and UNRESOLVE from artifacts only.
6. Mark any useful check that goes beyond the native claim as stronger_check.
7. Do not infer outcomes and do not use agent identity, traces, or scores.
```

C UNRESOLVE Reason Taxonomy

A record labeled UNRESOLVE receives exactly one upstream blocking reason. The labels are designed to make missing evidence actionable for benchmark authors; they are not meant to be a universal ontology.

Table 10: UNRESOLVE-reason labels. The upstream-priority rule assigns the reason that, if resolved first, would unblock the other missing evidence.

Reason	What is missing	Representative example
R1: Missing state query	The benchmark claim depends on a backend or environment state value that was never queried or stored.	A database task has no post-run read of the relevant table.
R2: Unobservable side effect	The action is performed, but delivery, send, payment, receipt, or another side effect is not exposed in any artifact.	A message appears to be sent, but the tool returns no receipt or message id.
R3: Ambiguous identity mapping	Multiple records or recipients match the claim and the trace does not identify which one was acted on.	Two customers share the same name and no unique id is logged.
R4: Missing state-preservation evidence	The native task or policy requires preserving unrelated state, but no snapshot or diff can check it.	A calendar edit requires preserving other events, but no post-run diff is captured.
R5: Paired-arm asymmetry	A paired case requires both arms to be decidable, but one arm is decidable and the other is not.	The benign arm is verifiable while the injected arm's security claim is not.
R6: Evaluator-output ambiguity	The released evaluator emits a label, but its stored inputs do not uniquely support that label under the benchmark claim.	A verifier accepts a string match without evidence that the structured target state was reached.
R7: Claim-scope mismatch	The trace supports a related task, but not the benchmark claim being reported.	The agent gives a helpful refund response, but the policy-relevant backend update required by the claim is absent.

D Additional Evaluation Tables

The main text keeps only the tables needed to follow the argument. This section holds the longer tables that will be filled directly from the scored manifest.

Table 11: Per-agent evidence-supported bounds by benchmark. Each cell reports native score, counted-only score, lower endpoint, upper endpoint, and width.

Benchmark	OpenAI GPT-5.4	Claude 4.6	DeepSeek V4 Pro
AGENTDOJO	FILL	FILL	FILL
APPWORLD	FILL	FILL	FILL
WEBARENA-VERIFIED	FILL	FILL	FILL
τ^3-bench retail	FILL	FILL	FILL

Table 12: Descriptive case-resampling intervals for evidence-supported quantities. These intervals condition on stored traces and fixed agent versions; they do not include repeated-execution randomness, API drift, tool nondeterminism, or environment volatility.

Domain	Lower endpoint	Upper endpoint	Width	Counted-only score	Native score inside bounds
AGENTDOJO	FILL	FILL	FILL	FILL	FILL
APPWORLD	FILL	FILL	FILL	FILL	FILL
WEBARENA-VERIFIED	FILL	FILL	FILL	FILL	FILL
τ^3-bench retail	FILL	FILL	FILL	FILL	FILL
All domains	FILL	FILL	FILL	FILL	FILL

E Audit, Rerun, and Adoption Cost

The audit checks whether the evidence labels are reproducible under human inspection. The rerun check is narrower: it probes label stability for a fixed reference model on a small repeated-execution

Table 13: Full unresolved-reason distribution. The main text reports the top reason per benchmark; this table exposes the complete reason histogram.

Benchmark	R1	R2	R3	R4	R5	R6	R7	Interpretation
AGENTDOJO	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
APPWORLD	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
WEBARENA-VERIFIED	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
τ^3-bench retail	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL

Table 14: Pairwise dominance margins for bound-separated comparisons. For a reported ordering $i > j$, the margin is $M_{i>j} = \mathrm{LB}_{i,d} - \mathrm{UB}_{j,d}$. The bootstrap interval is a descriptive stability check, not a formal significance test.

Domain	Pair	Bound decision	Margin and case-resampling interval	Interpretation
FILL FROM SCORED MANIFEST	–	–	–	Include every bound-separated pair and its case-cluster bootstrap dominance margin.

subset. Neither check is used to change the denominator or to estimate repeated-execution performance uncertainty.

Table 15: Audit protocol. The final report should fill the sampled counts and agreement statistics from the audit spreadsheet.

Protocol item	Planned reporting
Audit sample	25 records per benchmark, stratified across counted records, unresolved records, and native/evidence disagreements.
Blinding	Auditors see task text, trace, artifacts, and locked checklist; they do not see model identity, native label, evidence label, or reason label.
Primary agreement	Three-way label agreement and countability κ.
Reason agreement	`UNRESOLVE`-reason κ, computed only when both auditors mark the record unresolved.
Rerun subset	OpenAI GPT-5.4 on ten case units per benchmark; report evidence-label agreement and dominant disagreement pattern.
Resolution rule	Disagreements trigger checklist clarification or reason-label correction; they are logged rather than silently overwritten.

F Release and Result Files

Public release should make denominators and bounds auditable without exposing live credentials, real third-party keys, or side-effecting services.

- **Released publicly.** Case ids, locked case checklists, native claim, stronger-check flag, native label, evidence label, reason code, bound contribution, denominator audit, aggregate tables, plotting scripts, and audit instructions.
- **Released under access control when needed.** Full traces that contain prompt-injection content, synthetic personal data, or credential-shaped strings after scrubbing and sandbox re-pinning.
- **Not released.** Live credentials, real API keys, or artifacts that would reproduce side effects against non-sandboxed services.
- **Versioning.** Reported bounds are paired with a version tuple for the scorer, manifest, checklist template, checklist-generation prompt, and `UNRESOLVE` taxonomy.

The camera-ready build should be produced from the scored manifest. The file `outputs/latex/results_macros.tex` should set `\resultdatatrue` and define the domain-level macros used in Tables 2–4. The separate row files `per_agent_rows.tex`,

Table 16: Approximate human-time cost of adopting the method on top of an existing benchmark. Times are wall-clock minutes from trained annotators and exclude execution compute.

Domain	Draft/fix checklist	Score evidence	Tag unresolved	Setup note
AGENTDOJO	FILL	FILL	FILL	paired-arm files
APPWORLD	FILL	FILL	FILL	unit tests/db
WEBARENA-VERIFIED	FILL	FILL	FILL	verifier state
τ^3-bench retail	FILL	FILL	FILL	policy/id fields

`pairwise_margin_rows.tex`, `unresolve_reason_rows.tex`, and `cost_rows.tex` populate the longer appendix tables. Missing values should remain visibly marked as FILLrather than replaced by guessed numbers.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes].

   Justification: The abstract and introduction state the reporting method, the fixed four-domain study, and the intended scope. The discussion explicitly limits the claims to evidence-supported reporting rather than benchmark validity or a new leaderboard.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes].

   Justification: The Discussion section lists limitations of the locked claim, taxonomy scope, four-agent design, instrument-level `UNRESOLVE`, and bootstrap interpretation.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A].

Justification: The paper defines a reporting object and counting rule but does not present formal theorem statements or proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes].

Justification: Sections on study design, evidence-contract locking, denominator rules, audit checks, and the result-macro contract describe how the reported quantities are produced from the released manifests and rescorer.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes].

   Justification: The paper states that manifests, evidence contracts, case cards, taxonomy records, and a rescorer are released; the responsible release plan specifies which artifacts are public and which are access-controlled.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes].

   Justification: The paper specifies domains, case-unit selection, agents as fixed probes, denominator handling, contract drafting and locking, resampling, audit, and rerun checks.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes].

   Justification: The paper reports case-cluster bootstrap intervals as descriptive resampling diagnostics and explains their assumptions and limitations in Sections 3 and 3.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No].

   Justification: The appendix reports human-time adoption costs, but the current manuscript does not yet give full hardware, memory, wall-clock execution, or total compute details for every benchmark run.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes].

   Justification: The work uses public or benchmark-provided artifacts and includes a responsible release plan for traces, synthetic personal data, credentials, and access-controlled materials.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes].

Justification: The Discussion and Responsible Release Plan describe benefits for benchmark reporting and risks around state-changing workflows, security-related traces, synthetic personal data, and operational details.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [Yes].

Justification: The Responsible Release Plan separates fully public artifacts, access-controlled traces, and non-released credentials or operational artifacts.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No].

Justification: The manuscript cites the existing benchmarks used in the study, but the current draft does not yet enumerate every asset license and terms-of-use constraint in one place.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.

- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes].

    Justification: The paper documents the released manifests, evidence contracts, taxonomy tags, rescorer, denominator audits, and versioning in the E&D-oriented release and responsible release sections.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A].

    Justification: The blinded audit is an expert audit of benchmark records and evidence labels, not a crowdsourcing experiment or a study of human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A].

Justification: The work does not collect personal data from human subjects or run a human-subjects experiment; the audit concerns benchmark traces and evidence-label decisions.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes].

Justification: The paper describes the contract-drafting LLM, its inputs, its exclusion from agent traces and outcomes, and the human locking process before scoring.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.